

Concepts à rechercher — documentation détaillée

Analyse exploratoire

Statistiques descriptives Indicateurs résumant une variable numérique : moyenne, médiane, écart-type, minimum, maximum, quartiles (Q1, Q2, Q3). Première photographie du dataset avant tout traitement.

Skewness Mesure la symétrie d'une distribution autour de sa moyenne. $Skew \approx 0$: distribution symétrique. $Skew > 0$: queue étalée à droite (valeurs extrêmes élevées, ex. **BALANCE**, **CASH_ADVANCE**). $Skew < 0$: queue étalée à gauche. Une forte asymétrie justifie une transformation avant standardisation ou clustering.

Méthode IQR (outliers) $IQR = Q3 - Q1$. Bornes de détection : $[Q1 - 1.5 \times IQR ; Q3 + 1.5 \times IQR]$. Toute valeur hors de cet intervalle est un outlier potentiel, visualisable via un boxplot.

Corrélation de Pearson Mesure la relation linéaire entre deux variables numériques, comprise entre -1 et 1. Proche de 1 ou -1 : forte relation linéaire (positive ou négative). Proche de 0 : pas de relation linéaire détectée. Utile pour repérer des redondances avant PCA.

Histogramme et KDE Histogramme : répartition des valeurs en intervalles (bins). KDE (Kernel Density Estimation) : courbe de densité lissée superposée, pour une lecture continue de la forme de la distribution et de son asymétrie.

Prétraitement

Transformation logarithmique (log1p**)** Applique $\log(1 + x)$ à une variable positive et asymétrique, pour compresser les grandes valeurs et étaler les petites. **log1p** évite l'erreur mathématique de $\log(0)$. Toujours appliquée avant la standardisation.

Standardisation vs normalisation Standardisation (**StandardScaler**) : $(x - \text{moyenne}) / \text{écart-type}$, résultat moyenne 0 / écart-type 1 — adaptée à K-means, PCA, SVM. Normalisation (**MinMaxScaler**) : $(x - \text{min}) / (\text{max} - \text{min})$, valeurs entre 0 et 1, plus sensible aux outliers.

PCA Technique de réduction de dimension transformant des variables corrélées en composantes principales non corrélées, ordonnées par quantité d'information (variance) capturée.

Variance expliquée / cumulée Variance expliquée : proportion d'information portée par chaque composante. Variance cumulée : somme progressive, utilisée pour décider combien de composantes conserver (ex. seuil 80 %).

Scree plot Courbe de la variance expliquée (ou cumulée) en fonction du nombre de composantes ; le point d'inflexion ("coude") indique souvent un bon compromis.

Clustering — K-means

Principe de K-means (centroïdes) Algorithme de partitionnement qui répartit les points en **k** groupes en minimisant la distance entre chaque point et le centroïde de son cluster, par itérations successives (assignation puis recalcul des centroïdes).

Hyperparamètres K-means

- `n_clusters` : nombre de clusters `k`, le plus déterminant.
- `init` : méthode d'initialisation des centroïdes (`k-means++` recommandé).
- `n_init` : nombre d'exécutions avec initialisations différentes ; le meilleur résultat est conservé.
- `max_iter` : nombre maximal d'itérations avant arrêt.
- `random_state` : graine aléatoire, pour la reproductibilité.

k-means++ Méthode d'initialisation qui éloigne les centroïdes de départ les uns des autres, réduisant le risque de convergence vers une mauvaise solution locale.

Méthode du coude Trace l'inertie (distance au carré point-centroïde) en fonction de `k`. On cherche le point où la baisse ralentit nettement, signe qu'ajouter des clusters n'apporte plus un gain proportionnel.

Score de silhouette Mesure, pour chaque point, sa cohésion avec son cluster par rapport aux clusters voisins. Compris entre -1 et 1 : proche de 1 = bien séparé, proche de 0 = à la frontière, négatif = probablement mal classé.

Clustering — DBSCAN

Principe de DBSCAN Algorithme basé sur la densité : regroupe les points densément connectés et isole les points isolés comme "bruit", sans fixer de nombre de clusters à l'avance.

Hyperparamètres DBSCAN

- `eps` : rayon maximal pour considérer deux points comme voisins.
- `min_samples` : nombre minimal de points requis dans le voisinage pour former une zone dense.
- `metric` : métrique de distance utilisée (euclidienne par défaut).

Points core / border / noise Core : point ayant au moins `min_samples` voisins dans un rayon `eps`. Border : dans le voisinage d'un core point mais sans assez de voisins propres. Noise : n'appartient à aucun cluster dense, étiqueté `-1`.

Indice de Davies-Bouldin Compare la dispersion interne de chaque cluster à la distance entre centres de clusters. Plus la valeur est basse, meilleure est la séparation.

Indice de Calinski-Harabasz Rapport entre dispersion inter-cluster et intra-cluster. Plus la valeur est élevée, meilleure est la séparation.

Interprétation des clusters

Profiling client / personas Traduction de statistiques (moyennes, seuils) en descriptions compréhensibles par un public non technique, en attribuant un nom et une narration à chaque segment.

Moyennes standardisées vs réelles par cluster Vue standardisée : compare les clusters sur une échelle commune, repère les variables les plus extrêmes. Vue réelle (valeurs non transformées) : nécessaire pour formuler des seuils métier compréhensibles.

Instabilité du label numérique en clustering Le numéro attribué par K-means à chaque cluster (0, 1, 2...) est arbitraire et peut changer d'un entraînement à l'autre, même à paramètres identiques. Il faut ré-analyser les

moyennes de son propre run avant d'attribuer des noms métier.

Classification — fondations

Split train/test stratifié Division du dataset en conservant les mêmes proportions de classes dans train et test (paramètre `stratify`). Essentiel si les classes sont déséquilibrées.

Déséquilibre de classes Situation où une ou plusieurs classes sont beaucoup moins représentées. Un modèle peut ignorer la classe minoritaire tout en gardant une bonne accuracy globale — d'où l'importance du rappel et du F1-score par classe.

SMOTE / oversampling / undersampling Undersampling : réduire la classe majoritaire. Oversampling : dupliquer/générer des exemples de la classe minoritaire. SMOTE : génère des exemples synthétiques par interpolation entre exemples existants (`imbalanced-learn`).

Pipeline scikit-learn Objet qui enchaîne plusieurs étapes (ex. `StandardScaler` puis classifieur) en un seul bloc, garantissant que les mêmes transformations sont appliquées à l'entraînement et à la prédiction.

Data leakage Erreur consistant à laisser une information du jeu de test influencer l'entraînement (ex. standardiser avant de séparer train/test), faussant les performances mesurées. Règle : le prétraitement s'apprend uniquement sur `X_train`.

Classification — modèles

Régression logistique et hyperparamètres Modèle linéaire estimant une probabilité de classe via une fonction sigmoïde. `C` : inverse de la régularisation. `penalty` : type de régularisation (`l1`, `l2`). `solver` : algorithme d'optimisation.

Arbre de décision et hyperparamètres Segmente les données par une série de règles binaires organisées en arbre. `max_depth` : profondeur maximale. `min_samples_split` : nombre minimal d'exemples pour diviser un nœud. `criterion` : mesure de qualité de split (`gini`, `entropy`).

Random Forest et hyperparamètres Ensemble d'arbres de décision entraînés sur des sous-échantillons aléatoires, agrégés par vote majoritaire. `n_estimators` : nombre d'arbres. `max_features` : nombre de variables considérées à chaque split.

Feature importance Mesure (`feature_importances_`) de la contribution de chaque variable aux prédictions d'un modèle comme Random Forest.

SVM et hyperparamètres Cherche l'hyperplan séparant au mieux les classes en maximisant la marge. `C` : tolérance aux erreurs de classification. `kernel` : fonction noyau (`linear`, `rbf`, `poly`) pour gérer des frontières non linéaires. `gamma` : influence de chaque point d'entraînement.

Classification — évaluation

Matrice de confusion Tableau croisant classes prédites et classes réelles, pour visualiser les erreurs de classification par classe.

Précision, rappel, F1-score Précision : parmi les prédictions positives, proportion réellement positive. Rappel : parmi les cas réellement positifs, proportion correctement détectée. F1-score : moyenne harmonique des deux, utile en cas de déséquilibre de classes.

Validation croisée / StratifiedKfold Divise les données en `k` folds, entraîne sur `k-1` et valide sur le fold restant, en répétant l'opération. `StratifiedKfold` conserve les proportions de classes dans chaque fold.

GridSearchCV / RandomizedSearchCV GridSearchCV : teste exhaustivement toutes les combinaisons d'une grille d'hyperparamètres. RandomizedSearchCV : teste un nombre limité de combinaisons tirées aléatoirement, plus rapide sur de grands espaces.

Sérialisation (joblib) Permet de sauvegarder (`dump`) et recharger (`load`) un objet Python entraîné (ex. pipeline complet), pour le réutiliser sans ré-entraînement.

MLflow

Experiment / Run Experiment : espace nommé regroupant tous les essais d'un même objectif. Run : une exécution individuelle avec ses propres paramètres, métriques et artefacts.

`log_param`, `log_metric`, `log_artifact` `log_param` : enregistre un hyperparamètre. `log_metric` : enregistre une valeur de performance. `log_artifact` : enregistre un fichier associé au run (image, rapport).

`mlflow.sklearn.log_model` Sauvegarde directement l'objet pipeline scikit-learn entraîné dans MLflow, pour le recharger et le réutiliser sans ré-entraînement.

MLflow UI Interface web (`mlflow ui`) affichant tous les runs sous forme de tableau triable, pour comparer visuellement les configurations testées.

Streamlit

Widgets Streamlit Composants d'interface : `st.file_uploader` (chargement de fichier), `st.number_input` (saisie numérique), `st.button` (déclenchement d'action), `st.write` (affichage de résultat).

`predict_proba` Méthode renvoyant, pour un modèle de classification, la probabilité d'appartenance à chacune des classes possibles plutôt qu'une seule classe.

Chargement d'un pipeline sérialisé Bonne pratique : charger dans l'application l'objet pipeline complet sauvegardé via `joblib`, plutôt que de recoder manuellement le prétraitement — garantit la cohérence entre entraînement et production.

Documentation et reproductibilité

Structuration modulaire d'un projet Python Organisation du code en fichiers spécialisés (`preprocessing.py`, `clustering.py`, `classification.py`, `pipeline.py`) plutôt qu'un notebook unique, pour faciliter réutilisation et maintenance.

`pyproject.toml` / `uv pyproject.toml` : fichier standard décrivant dépendances et configuration d'un projet Python. `uv` : gestionnaire de paquets rapide, avec fichier de verrouillage (`uv.lock`) garantissant des versions

figées entre machines.

README technique Document décrivant le projet, sa structure, les instructions d'installation et d'exécution, pour qu'un tiers puisse reprendre le travail.

Méthodologie Agile (Epics, tickets, estimation) Découpage d'un objectif large (Epic) en tâches concrètes (tickets), chacune estimée en charge de travail, pour planifier et suivre l'avancement.

Airflow (bonus)

DAG Représentation d'un pipeline sous forme de graphe orienté sans cycle, où chaque nœud est une tâche et chaque flèche une dépendance d'exécution.

Opérateurs Airflow Blocs prédéfinis représentant un type d'action (exécuter un script Python, une requête, etc.), utilisés pour construire les tâches d'un DAG.

Scheduling / retries Scheduling : fréquence d'exécution automatique d'un DAG. Retries : nombre de nouvelles tentatives automatiques en cas d'échec d'une tâche.

Interface Airflow Interface web pour visualiser l'état d'exécution des DAG, superviser les tâches en cours et consulter l'historique des exécutions.